PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL
FORMATO GUÍA DE APRENDIZAJE

Guía 4 DESARROLLO DE BASES DE DATOS DML Y DDL

1. IDENTIFICACIÓN DE LA GUIA DE APRENDIZAJE

- Denominación del Programa de Formación: Técnico en Programación de Software
- Código del Programa de Formación: 233104 versión 2
- Nombre del Proyecto Formativo: Construcción de sistema de información para emprendedores y empresarios.
- Fase del Proyecto: Diseño o Formulación del Proyecto (Planeación).
- Actividad de Proyecto Formativo: Administrar la base de datos de acuerdo a los estándares y requisitos técnicos del cliente.
- Competencia: Administrar la base de datos de acuerdo a los estándares y requisitos técnicos del cliente.
- Resultados de Aprendizaje: Construir la base de datos según requerimientos del cliente.
- **2. PRESENTACIÓN**



Las bases de datos son el corazón de la información en el mundo digital. Almacenan, organizan y gestionan grandes cantidades de datos de manera eficiente y segura. Si quieres comprender cómo funcionan las aplicaciones que utilizamos a diario, desde redes sociales hasta sistemas bancarios, es esencial conocer las bases de datos relacionales.

Esta guía te llevará paso a paso por los conceptos fundamentales del modelado de datos, el lenguaje de definición de datos (DDL) y el lenguaje de manipulación de datos (DML). Aprenderás a diseñar estructuras de bases de datos eficientes y a realizar consultas

poderosas para obtener la información que necesitas, trabajando sobre un caso real del sector agrícola que integra MySQL, Python en Google Colab y visión artificial con YOLO.

FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

Descripción de la actividad: En esta actividad vas a construir, paso a paso, la base de datos de una empresa agrícola colombiana que tiene fincas en Cundinamarca, Caldas y Boyacá. La empresa necesita registrar sus lotes, los insumos de la bodega (fertilizantes, fungicidas, insecticidas y bioinsumos) con los datos de su anexo técnico, las aplicaciones que se hacen en campo y las detecciones de plagas que realiza un dron con visión artificial (YOLO). Trabajarás con MySQL dentro de Google Colab y practicarás cada tema en el simulador web AgroDatos Lab.

Recursos de la actividad

- **Cuaderno de Google Colab** `AgroDatos_MySQL_Colab.ipynb`: ábrelo en Colab con *Archivo → Subir cuaderno*. Contiene el código base de los cinco puntos.
- **Simulador AgroDatos Lab** (enlace entregado por el instructor): cinco laboratorios interactivos con una base de datos real en el navegador.
- **Cuenta de GitHub** para entregar las evidencias.

Indicaciones generales

- Puedes trabajar en parejas. Escriban los nombres de los integrantes en la primera celda del cuaderno.
- Ejecuten las celdas en orden con *Shift* + *Enter*. Comiencen siempre por el **Ejercicio 0**, que instala el servidor MySQL. Si Colab se reinicia, vuelvan a ejecutarlo.
- Las celdas marcadas con **RETO** son las que ustedes deben programar. Las preguntas de reflexión se responden en las celdas de texto del mismo cuaderno.
- Los valores de dosis, carencia, reingreso y registro ICA son didácticos. En una finca real siempre se consulta la etiqueta y la ficha técnica aprobada por el ICA.

PRIMER PUNTO: Modelado y creación de tablas (DDL)

"*Más vale prevenir que lamentar*" - Dicho popular

En una base de datos, prevenir significa diseñar bien las tablas antes de guardar el primer dato. Si una tabla permite registrar un lote con área negativa o dos lotes con el mismo código, el error aparecerá meses después en un reporte equivocado o en una cosecha mal planeada. Las restricciones (PRIMARY KEY, NOT NULL, UNIQUE, CHECK) hacen que sea el motor de base de datos, y no la memoria del programador, quien proteja la calidad de la información.

Objetivo: pasar de la descripción de un negocio a un modelo relacional y crearlo en MySQL con sentencias DDL (*Data Definition Language*).

Situación: *La empresa tiene varias fincas, cada una ubicada en un municipio y un departamento, a una altitud determinada. Cada finca se divide en lotes; un lote tiene un código único (por ejemplo ESP-L01), un cultivo sembrado, un área en hectáreas y una fecha de siembra.*

Conceptos clave

Restricción	Qué garantiza	Ejemplo en el cultivo
PRIMARY KEY	Cada fila se identifica de forma única y no es nula	id_lote
NOT NULL	El dato es obligatorio	Todo lote debe tener área
UNIQUE	El valor no se repite	No pueden existir dos lotes ESP-L01
CHECK	El valor está dentro del dominio permitido	area_ha > 0
DEFAULT	Valor que se usa si no se envía ninguno	estado = 'activo'

Actividades

1. En el cuaderno de Colab ejecuta el **Ejercicio 0** y verifica que la última celda muestre la versión de MySQL y la hora del servidor. Toma una captura de pantalla.
2. A partir de la situación descrita, identifica las **entidades**, sus **atributos** y la **relación** entre ellas. Dibuja el modelo entidad-relación (a mano, en draw.io o en el Laboratorio 1 del simulador) indicando la cardinalidad finca–lote.
3. Ejecuta las celdas 1.1 a 1.3 para crear la base de datos `agro_campo` con las tablas `finca` y `lote` e insertar los datos de ejemplo. Explica con tus palabras para qué sirve cada restricción de la tabla `lote`.
4. Ejecuta las celdas 1.4, 1.5 y 1.6, que deben fallar. Copia el código y el mensaje de cada error, e indica qué restricción lo produjo.
5. **RETO 1.** Resuelve en la celda indicada:
   a. Agrega a `lote` la columna `tipo_riego` con los valores `'goteo'`, `'aspersión'`, `'gravedad'` y `'secano'`, con valor por defecto `'secano'` (usa `ALTER TABLE … ADD COLUMN`).
   b. Crea la tabla `operario` con `id_operario` (llave primaria autoincremental), `documento` (obligatorio y único), `nombre_completo` (obligatorio), `telefono` y `certificado_aplicador` (BOOLEAN, falso por defecto).
   c. Inserta tres operarios y muestra la tabla con `SELECT`.
6. **Práctica en el simulador:** en el **Laboratorio 1 · Modelo relacional** revisa las siete tablas del sistema y resuelve la actividad de cardinalidades hasta obtener 5 de 5 correctas.

Reflexión

7. ¿Por qué `area_ha` se define como `DECIMAL(6,2)` y no como `FLOAT`? ¿Qué problema de redondeo se evita?
8. ¿Qué diferencia hay entre validar el área en el formulario HTML de la página web y validarla con `CHECK` en MySQL? ¿Por qué conviene hacer ambas cosas?
9. Si mañana la empresa empieza a sembrar arroz, ¿qué desventaja tiene haber usado `ENUM` para el cultivo? ¿Qué alternativa propondrías?

SEGUNDO PUNTO: Integridad referencial e insumos con anexo técnico

"*Cuentas claras, amistades largas*" - Dicho popular

En la bodega de una finca, una cuenta poco clara puede costar dinero o la salud de un trabajador. Si se registra mal la categoría toxicológica de un fungicida o su periodo de reingreso, alguien puede entrar al lote antes de tiempo. Las llaves foráneas mantienen las cuentas claras entre tablas: no puede existir una aplicación de un producto que no está registrado, ni en un lote que no existe.

Objetivo: relacionar tablas con llaves foráneas (FOREIGN KEY) y registrar productos agrícolas a partir de su anexo técnico, comprobando que los datos sean válidos y además fieles al documento.

El anexo técnico de un insumo

Cada producto trae una ficha técnica. Estos son los datos que la base de datos debe guardar con exactitud:

- **Tipo:** fertilizante, enmienda, fungicida, insecticida, herbicida o bioinsumo.
- **Ingrediente activo** y su concentración.
- **Categoría toxicológica:** I (franja roja, extremadamente tóxico), II (amarilla, altamente tóxico), III (azul, medianamente tóxico) y IV (verde, ligeramente tóxico).
- **Registro ICA:** número único de registro de venta.
- **Dosis por hectárea** y su unidad (kg, L, g, mL).
- **Periodo de carencia (días):** tiempo mínimo entre la última aplicación y la cosecha. Protege al consumidor.
- **Periodo de reingreso (horas):** tiempo antes de que una persona pueda volver a entrar al lote. Protege al trabajador.

Anexos para registrar (valores didácticos):

Nombre comercial	Tipo	Ingrediente activo	Cat.	Registro ICA	Dosis/ha	Car. (días)	Reing. (horas)	Precio COP
NitroCampo 46	fertilizante	Urea 46 % N	IV	ICA-DEMO-1001	150 kg	0	0	3.800
FosfoArranque	fertilizante	Fosfato diamónico 18-46-0	IV	ICA-DEMO-1002	100 kg	0	0	4.500

Nombre comercial	Tipo	Ingrediente activo	Cat.	Registro ICA	Dosis/ha	Car. (días)	Reing. (horas)	Precio COP
CalViva	enmienda	Cal dolomita	IV	ICA-DEMO-1003	500 kg	0	0	600
BioLarva WP	bioinsumo	Bacillus thuringiensis var. kurstaki	IV	ICA-DEMO-2001	0,5 kg	0	4	95.000
RoyaStop 250 EC	fungicida	Triazol 250 g/L	III	ICA-DEMO-3001	0,4 L	30	24	160.000
TizónFin 80 WP	fungicida	Ditiocarbamato 800 g/kg	III	ICA-DEMO-3002	2,5 kg	14	24	38.000

Actividades

1. Ejecuta la celda 2.1–2.2. Explica qué hace cada `CONSTRAINT … FOREIGN KEY` y qué significa `ON DELETE RESTRICT`. Dibuja la relación muchos a muchos entre **lote** y **producto** resuelta con la tabla intermedia **aplicacion**.
2. Ejecuta la celda 2.3, que registra los tres primeros productos del anexo.
3. **RETO 2A.** Registra en la tabla `producto` los tres productos faltantes (BioLarva WP, RoyaStop 250 EC y TizónFin 80 WP) copiando con exactitud los datos del anexo. Muestra el resultado con `SELECT`.
4. Ejecuta la celda 2.4 para registrar aplicaciones y luego la celda 2.5, que debe fallar. Copia los dos mensajes de error y explica qué protegió el motor en cada caso.
5. **RETO 2B.** Resuelve en la celda indicada:
   a. Registra una aplicación de **RoyaStop 250 EC** en el lote de café `POR-L01`, calculando la cantidad como *dosis del anexo* × *área del lote* con una subconsulta (sin escribir el número a mano).
   b. Escribe una consulta que muestre, para cada aplicación, la fecha mínima de cosecha permitida usando `DATE_ADD(fecha, INTERVAL carencia_dias DAY)`.
6. **Práctica en el simulador:** en el **Laboratorio 2 · Bodega de insumos** registra al menos cuatro productos pendientes leyendo su anexo. Provoca a propósito un error de `CHECK` (dosis negativa) y uno de `UNIQUE` (registro ICA repetido) y toma captura de cada mensaje. Después registra una aplicación y anota la fecha de reingreso y de cosecha que calcula el sistema.

Reflexión

7. ¿Qué error se produjo al insertar una aplicación en el lote 99? ¿Qué pasaría en una página web si esa llave foránea no existiera?
8. ¿Por qué no se pudo borrar la finca 1? Describe un caso del negocio en el que sí usarías `ON DELETE CASCADE`.
9. En el simulador, el motor aceptó un registro con la categoría toxicológica equivocada. ¿Por qué ninguna restricción lo detectó? ¿Qué controles agregarías en la aplicación web para evitarlo?

10. Un operario entra al lote 10 horas después de aplicar RoyaStop. ¿Qué dato de la base de datos debió impedirlo y cómo lo mostrarías en la página web?

TERCER PUNTO: Consultas para tomar decisiones (DML)

"*Preguntando se llega a Roma*" - Dicho popular

Los datos guardados solo sirven si sabemos hacerles preguntas. El agrónomo no quiere ver tablas: quiere saber qué lote le está costando más por hectárea, dónde se aplicó de más y qué lotes llevan meses sin fertilizar. Cada una de esas preguntas se traduce en una consulta SQL.

Objetivo: responder preguntas del negocio combinando varias tablas con `SELECT`, `WHERE`, `JOIN`, `GROUP BY` y `HAVING`.

Conceptos clave

- `INNER JOIN` muestra solo las filas que tienen pareja en ambas tablas; `LEFT JOIN` conserva todas las filas de la tabla izquierda, aunque no tengan pareja.
- `GROUP BY` agrupa filas para aplicar funciones de agregado: `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`.
- `WHERE` filtra filas antes de agrupar; `HAVING` filtra grupos después de agrupar.
- Orden lógico de evaluación: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT.

Actividades

1. Ejecuta las consultas 3.1 a 3.4. Para cada una escribe, en una línea, qué pregunta del agrónomo responde.
2. En la consulta 3.4 identifica si hay alguna aplicación con **SOBREDOSIS** o **SUBDOSIS**. Explica qué cálculo hace la expresión `CASE`.
3. **RETO 3.** Escribe una consulta para cada pregunta:
   a. Área total sembrada por cultivo, ordenada de mayor a menor.
   b. Fincas cuya área total sembrada supera 5 ha (usa `HAVING`).
   c. Productos que nunca se han aplicado.
   d. El responsable que más kilogramos de fertilizante ha aplicado.
4. **Práctica en el simulador:** en el **Laboratorio 4 · Consola SQL** resuelve los seis retos hasta que el sistema marque cada uno como correcto. Prueba también los botones de ejemplo *Error de FK* y *Error de CHECK* y compara los mensajes con los que obtuviste en MySQL.

Reflexión

5. En la consulta 3.3, ¿qué pasaría si cambias `LEFT JOIN` por `JOIN`? Explica por qué.
6. ¿Por qué el `GROUP BY` de la consulta 3.2 incluye `l.codigo` y `l.cultivo` además de `l.id_lote`?

7. Si una página web muestra un tablero con el costo por hectárea, ¿conviene calcularlo en SQL o en JavaScript del navegador? Argumenta pensando en la cantidad de datos transferidos y en la seguridad.

CUARTO PUNTO: Normalización de una planilla de campo

"*Cada cosa en su lugar y un lugar para cada cosa*" - Dicho popular

Muchas fincas llevan sus registros en una sola hoja de Excel. Al pasarla tal cual a una tabla, el teléfono del operario se repite en cada fila, el nombre de un producto se escribe de tres formas y, si se borra la única aplicación de un producto, desaparece todo lo que se sabía de él. Normalizar es poner cada dato en su lugar: en una sola tabla y una sola vez.

Objetivo: detectar las anomalías de una tabla sin normalizar y llevarla hasta la Tercera Forma Normal (3FN).

Conceptos clave

Concepto	Definición	Ejemplo en la planilla
Anomalía de actualización	El mismo dato está repetido y al cambiarlo se olvida alguna copia	El operario 1012 aparece con dos teléfonos
Anomalía de inserción	No se puede registrar algo sin un dato que aún no existe	No se puede crear un operario sin una aplicación
Anomalía de borrado	Al borrar una fila se pierde información de otra entidad	Se borra la única aplicación de un producto
1FN	Valores atómicos, sin listas ni grupos repetidos	Observaciones con varios datos en un texto
2FN	1FN y cada atributo depende de toda la llave	categoria_tox depende del producto
3FN	2FN y sin dependencias transitivas	documento → teléfono del operario

Actividades

1. Ejecuta la celda 4.1 y analiza la planilla. Identifica y escribe un ejemplo concreto de anomalía de **actualización**, de **inserción** y de **borrado**.
2. Ejecuta la consulta 4.2 y explica qué detecta y por qué es útil para auditar datos.
3. Ejecuta la celda 4.3, que crea la tabla `operario_n`. Explica qué dependencia transitiva se elimina.
4. **RETO 4.** Resuelve en la celda indicada:
   a. Crea la tabla `aplicacion_n` en 3FN con `id`, `fecha`, `id_lote` (FK a lote), `id_producto` (FK a producto), `cantidad`, `documento_operario` (FK a operario_n) y `observaciones`.
   b. Migra los datos de la planilla con **un solo** `INSERT … SELECT` que haga `JOIN` con `lote` (por código) y con `producto` (por nombre).

c. Escribe la consulta que reconstruya la planilla original a partir de las tablas normalizadas.

5. Dibuja el modelo entidad-relación final (antes y después de normalizar) y súbelo al repositorio.

Reflexión

6. La columna `categoria_tox` de la planilla, ¿de qué depende realmente: de la aplicación o del producto? ¿Qué forma normal violaba?
7. La columna `observaciones` contiene textos como *cogollero en 30% de plantas*. ¿Cumple la 1FN? ¿Cómo la rediseñarías para poder consultar cuántas plantas tenían plaga?
8. ¿Cuándo es aceptable desnormalizar a propósito? Piensa en un reporte de una página web que se consulta miles de veces al día.

QUINTO PUNTO: Visión artificial con YOLO y MySQL

"*El ojo del amo engorda el ganado*" - Dicho popular

Hoy el ojo del amo puede ser un dron. Mientras sobrevuela el cultivo, un modelo de inteligencia artificial llamado YOLO revisa cada foto y marca dónde hay gusanos, roya, tizón o maleza. Un solo vuelo produce cientos de detecciones; sin una base de datos bien diseñada no se puede saber qué lote supera el umbral de daño, si la plaga avanza o qué producto aplicar y en qué cantidad.

Objetivo: almacenar en MySQL las detecciones de un modelo YOLO y convertirlas, con vistas, triggers y consultas, en una recomendación agronómica.

¿Qué es YOLO?

YOLO (*You Only Look Once*) es una familia de redes neuronales de **detección de objetos**. Un clasificador responde *qué hay* en una foto; un detector responde *qué hay y dónde está*. YOLO procesa la imagen completa en una sola pasada, por eso sirve para video de drones en tiempo real. Por cada objeto entrega:

Campo	Significado
clase	Qué objeto es: gusano_cogollero, roya, tizon, maleza…
confianza	Probabilidad estimada (0 a 1) de que la caja sea correcta
x, y, w, h	Centro, ancho y alto de la caja, normalizados entre 0 y 1
Umbral de confianza	Valor mínimo para aceptar una caja; filtra falsos positivos
IoU y NMS	IoU = intersección ÷ unión de dos cajas. NMS elimina cajas duplicadas sobre el mismo objeto

Actividades

1. Ejecuta la celda 5.1 y dibuja el modelo **lote** → **imagen** → **deteccion** y **clase_objetivo** → **producto**. Explica por qué la detección no guarda el código del lote.

2. **YOLO real.** Ejecuta las celdas 5.2 y 5.3. Primero usa la imagen de ejemplo y luego cambia `SUBIR_FOTO = True` para subir una foto propia (frutas, hortalizas o plantas). Copia la tabla de detecciones guardada en MySQL y comenta si el modelo acertó.
3. **Modelo del cultivo.** Ejecuta las celdas 5.4 y 5.5, que simulan 30 vuelos de dron con un modelo entrenado para plagas. Verifica cuántas imágenes y detecciones quedaron guardadas.
4. Ejecuta la vista 5.6 y la consulta 5.7. Indica qué lotes superan el umbral de daño, qué producto se sugiere y qué cantidad se debe sacar de la bodega.
5. Ejecuta la celda 5.8 y explica, línea por línea, qué hace el `TRIGGER trg_alerta_plaga`.
6. **RETO 5.** Resuelve en la celda indicada:
    a. **Tendencia:** cuenta las detecciones válidas de plaga por lote y por semana (`YEARWEEK`). ¿Algún lote empeora?
    b. **Densidad:** calcula las detecciones válidas de plaga por hectárea en cada lote.
    c. **Calidad del modelo:** muestra cuántas detecciones de maleza quedarían con umbrales de 0,30; 0,45; 0,60 y 0,75.
    d. **Diseño:** propone el `CREATE TABLE` o `ALTER TABLE` necesario para comparar el modelo actual con una nueva versión `yolo11n-agro-v2` sobre las mismas imágenes.
7. **Práctica en el simulador:** en el **Laboratorio 3 · Dron + YOLO** realiza tres vuelos sobre lotes de cultivos diferentes. En cada vuelo recorre las cuatro etapas (cuadrícula, candidatas, umbral y NMS), activa la verdad de campo y registra en una tabla los valores de precisión y recall con umbral de confianza 0,25; 0,45 y 0,80. Guarda las detecciones, anota la recomendación agronómica y responde las cuatro preguntas sobre YOLO.

Reflexión

8. ¿Qué diferencia hay entre clasificación y detección de objetos? ¿Por qué para contar gusanos por planta se necesita detección?
9. Al subir el umbral de confianza de 0,25 a 0,80, ¿qué pasa con los falsos positivos y los falsos negativos? Para el gusano cogollero, ¿qué error es más costoso para el agricultor? Apóyate en los datos que registraste en el simulador.
10. Explica con un ejemplo de dos cajas sobre la misma hoja con roya qué es el IoU y cómo lo usa el NMS.
11. ¿Por qué las coordenadas se guardan normalizadas (0 a 1) y no en píxeles?
12. En la celda 5.3 se usaron sentencias parametrizadas (`%s`). ¿Qué ataque evitan y por qué son obligatorias en el backend de una página web?
13. El modelo COCO detectó *broccoli* en un lote de papa. ¿Qué nos dice esto sobre usar modelos preentrenados fuera de su dominio?

EVALUACIÓN: Evidencias y criterios

Al terminar los cinco puntos presenta la **Evaluación final** (Laboratorio 5 del simulador) y toma una captura del puntaje. El mínimo para aprobar es 7 de 12.

Criterio	Peso	Excelente	Aceptable	Por mejorar
Retos SQL de los puntos 1 a 5	40 %	Todos ejecutan sin error y responden la pregunta	3 o 4 retos correctos	Menos de 3 retos
Restricciones e integridad	15 %	Justifica cada PK, FK y CHECK	Las usa sin justificar	Omite restricciones
Normalización	15 %	3FN completa y planilla reconstruida	Llega a 2FN	Tabla sin normalizar
YOLO y datos	15 %	Relaciona umbral, IoU y decisión agronómica con datos propios	Explica los conceptos sin relacionarlos	Respuestas incompletas
Reflexiones y simulador	15 %	Argumentadas con ejemplos del cultivo y capturas	Respuestas cortas correctas	Sin responder

NOTA: Desarrollar un repositorio de GITHUB denominado “Actividad 8” que contenga: el cuaderno `.ipynb` resuelto (*Archivo → Descargar → Descargar .ipynb*), los diagramas entidad-relación, las capturas del simulador y un archivo `README.md` con los nombres de los integrantes y el resumen de cada punto.

Actividad 1: Ejecución del Ejercicio 0

- ***Qué debes hacer: Ve a tu cuaderno de Colab, busca la celda del "Ejercicio 0" y dale al botón de Play (ejecutar). Verás que abajo se imprime un texto con la versión de MySQL instalada y la hora actual del servidor.***
- ***Tu acción: Toma una captura de pantalla completa de esa celda y su resultado para pegarla en tu reporte.***

Actividad 2: Identificación de Entidades y Modelo Entidad-Relación

Basándonos en la descripción de la primera parte, aquí tienes la estructura lógica para armar tu diagrama:

- ***Entidades y Atributos:***
  - ***Finca:*** `id_finca` ***(Clave Primaria),*** `nombre_finca`, `municipio`, `departamento`, `altitud`.
  - ***Lote:*** `id_lote` ***(Clave Primaria),*** `codigo_lote`, `cultivo_sembrado`, `area_ha`, `fecha_siembra`, `id_finca` ***(Clave Foránea).***

- ***Relación y Cardinalidad:***
  - ***Una Finca puede tener muchos (N) lotes.***
  - ***Un Lote pertenece de forma obligatoria a una (1) sola finca.***
  - ***Cardinalidad: 1 a N (Uno a Muchos) desde Finca hacia Lote.***

  💡 Tip para draw.io: Dibuja un rectángulo para Finca y otro para Lote. Únelos con una línea donde el lado de Finca tenga un "1" (o una línea simple) y el lado de Lote tenga una "N" (o una pata de gallo).

Actividad 3: Explicación de las Restricciones de la tabla `lote`

Cuando ejecutes las celdas 1.1 a 1.3, se creará la tabla `lote`. En tus propias palabras, puedes explicar sus restricciones así:

- ***PRIMARY KEY (`id_lote`): Es la cédula de identidad del lote. Asegura que ningún lote tenga el mismo identificador numérico interno y que este campo nunca se quede vacío.***
- ***FOREIGN KEY (`id_finca`): Es el cable que une el lote con su finca correspondiente. Sirve para que no se pueda registrar un lote en una finca fantasma o inexistente.***
- ***NOT NULL (en campos como área o cultivo): Obliga al usuario a escribir un dato real. Evita que un lote se guarde sin saber qué se sembró o cuánto mide.***
- ***UNIQUE (`codigo_lote`): Protege al negocio de confusiones lógicas. Garantiza que códigos de terreno como `ESP-L01` pertenezcan a un único espacio físico y no se dupliquen.***
- ***CHECK (`area_ha > 0`): Es un filtro de cordura matemática. Impide que por un error de digitación se guarden lotes con tamaños absurdos como 0 hectáreas o áreas negativas.***

Actividad 4: Análisis de Errores (Celdas 1.4, 1.5 y 1.6)

Como no puedo ver tu pantalla, aquí te describo exactamente qué causa cada error común en este tipo de ejercicios de bases de datos para que los asocies con tus resultados:

1. ***Error por Clave Duplicada (Duplicate entry): Ocurre si la celda intenta insertar un lote con un `codigo_lote` que ya existe. La restricción que lo produce es UNIQUE.***
2. ***Error por Valor Nulo (Column cannot be null): Ocurre si la celda intenta dejar vacío un campo obligatorio (como el área o el cultivo). La restricción involucrada es NOT NULL.***

3. ***Error de Validación Física (Check constraint violated): Ocurre si la celda intenta registrar un lote con un área menor o igual a cero (ej. `-2`). La restricción que salta es el CHECK.***

9. Categoría toxicológica equivocada aceptada por el motor

- ***¿Por qué ninguna restricción lo detectó? Porque el campo de la categoría toxicológica probablemente fue definido con un tipo de datos genérico (como `VARCHAR` o `TEXT`) y no se le configuró una restricción de validación (como un `CHECK` o un tipo de dato `ENUM`). Para el motor de la base de datos, la palabra "Extremadamente Peligroso" y la palabra "Frutica" son igual de válidas si ambas son texto.***
- ***Controles a agregar en la aplicación web:***

1. ***En la interfaz (Frontend): Cambiar el campo de texto libre por un elemento de selección cerrada, como un menú desplegable (Select / Dropdown) o botones de radio, obligando al usuario a elegir únicamente entre las categorías oficiales (I, II, III, IV).***

10. Ingreso no seguro de un operario al lote

- **¿Qué dato debió impedirlo?** El campo **Periodo de Reingreso (PR)** del agroquímico aplicado, calculado en la base de datos como la **Fecha/Hora de Reingreso**. Si el producto exige 24 horas de espera y solo han pasado 10, el sistema detecta que el lote se encuentra en estado de "Periodo de Carencia o Bloqueo".
- **¿Cómo lo mostrarías en la página web?**
  - **Alertas visuales de semáforo:** Un banner o tarjeta de color **rojo encendido** en el panel principal del lote con un ícono de advertencia (⚠).
  - **Mensaje de bloqueo dinámico:** Un texto que diga de forma explícita: *"PROHIBIDO EL INGRESO: Lote en periodo de reingreso por aplicación de RoyaStop. Seguro ingresar en X horas"*.
  - **Control de asistencia:** Si el administrador intenta asignarle una tarea en ese lote a un operario en la web, el botón de "Asignar" debe aparecer deshabilitado (gris) con un mensaje flotante de restricción médica.

1. Integridad Referencial y Relación Muchos a Muchos

Cuando ejecutas las celdas 2.1 y 2.2, se definen las restricciones de llave foránea en la tabla intermedia `aplicacion`.

- **¿Qué hace cada `CONSTRAINT ... FOREIGN KEY`?**

- `FOREIGN KEY (id_lote) REFERENCES lote(id_lote)`: Garantiza que no se registre una fumigación o fertilización en un lote fantasma. Cada aplicación debe apuntar estrictamente a un lote real de la finca.
- `FOREIGN KEY (id_producto) REFERENCES producto(id_producto)`: Garantiza que no se aplique un insumo químico que no haya sido registrado previamente en la bodega de productos.

- **¿Qué significa `ON DELETE RESTRICT`?**
- Es una directiva de máxima seguridad. Significa que **el sistema prohíbe tajantemente eliminar un lote o un producto si este ya tiene un historial de aplicaciones registrado**. Si intentas borrar el lote, el motor frena la acción para evitar que se pierda la trazabilidad legal y de salud de los químicos aplicados.

- **Diagrama de Relación Muchos a Muchos (M:N):**
  Un lote puede recibir muchas aplicaciones de diferentes productos a lo largo del año, y un mismo producto puede ser aplicado en muchos lotes distintos. Esta relación se rompe mediante la tabla intermedia `aplicacion`:

2. Ejecución de la celda 2.3

- **Acción:** Dale *Play* a la celda en tu cuaderno de Colab. Esta acción guardará los datos de *NitroCampo 46*, *FosfoArranque* y *CalViva* en tu base de datos local.

---

3. RETO 2A: Registro de productos faltantes (Código SQL)

Escribe y ejecuta este bloque en la celda indicada para completar tu inventario de insumos (recuerda que el ejercicio en tu Colab llama a la tabla `producto`):

4. Análisis de Errores (Celda 2.5)

Al ejecutar la celda 2.5 verás que el motor frena los registros de forma inmediata. Copia los textos de error de tu pantalla, los cuales se fundamentan en lo siguiente:

1. **Error de Llave Foránea Inexistente (Cannot add or update a child row: a foreign key constraint fails):** Ocurre si la celda intenta registrar una aplicación en un lote que no existe (como el lote 99).
   - *Qué protegió:* Evitó que se crearan registros huérfanos que arruinarían las estadísticas de costos y consumo por lote.
2. **Error de Restricción Check (Check constraint violated):** Ocurre si la celda intenta guardar datos absurdos, como un valor negativo en los días de carencia o en las dosis.

- *Qué protegió:* Protegió la coherencia matemática y legal del negocio agrícola; un producto no puede tener un tiempo de espera inverso o negativo antes de cosecharse.

---

5. RETO 2B: Subconsultas y Fechas de Cosecha (Código SQL)

a. Aplicación automatizada mediante subconsulta (Dosis × Área)

Este script busca de forma automática el área del lote `POR-L01` y la dosis recomendada de `RoyaStop 250 EC` directamente desde sus respectivas tablas, realizando la multiplicación matemática de manera interna sin que tengas que digitar ningún número:

6. Práctica en el Simulador

Sigue estos pasos dentro de la interfaz gráfica del Laboratorio 2:

1. ***Provocar CHECK: En la casilla de dosis de cualquier producto digita `-5`. Intenta guardar. Toma captura al aviso en rojo que te impide avanzar.***
2. ***Provocar UNIQUE: Registra un producto normal. Luego intenta crear otro diferente, pero en la casilla "Registro ICA" escribe idénticamente el código del anterior (`ICA-DEMO-1001`). Toma captura al error de duplicado.***
3. ***Cálculo de Fechas: Registra una aplicación real con la fecha de hoy. El simulador leerá las horas de reingreso y días de carencia para imprimir en pantalla las fechas seguras. Anota esas dos fechas exactas en tu informe escrito.***
   ¿Los nombres de las columnas en tu cuaderno de Colab coinciden exactamente (`carencia_dias`, `reingreso_horas`) o necesitas que adaptemos alguna consulta a una estructura específica de tu base de datos?

7. Error al insertar una aplicación en el lote 99

- ***¿Qué error se produjo? Se generó un error de violación de restricción de llave foránea (Foreign Key Constraint Violation). El motor de MySQL detuvo la operación porque se intentó registrar una bitácora de aplicación vinculada a un código de lote (`id_lote = 99`) que no existe en la tabla principal de lotes.***
- ***¿Qué pasaría en una página web si esa llave foránea no existiera? La aplicación web guardaría el registro con éxito, generando datos huérfanos.***

Con el tiempo, esto rompería el sistema: al generar el reporte de costos por lote, la aplicación web fallaría, se colgaría o arrojaría sumas misteriosas de dinero gastado en insumos que no pertenecen a ningún terreno real del negocio.

8. Intento de borrar la finca 1 y el uso de ON DELETE CASCADE

- ***¿Por qué no se pudo borrar la finca 1? Debido a la regla `ON DELETE RESTRICT` activa en la base de datos. Como la finca 1 tiene registros hijos (lotes asociados), el motor bloquea el borrado del registro padre para evitar que esos lotes queden sin una propiedad a la cual pertenecer.***
- ***Caso de negocio para usar `ON DELETE CASCADE`: Sería ideal aplicarlo en la tabla `monitoreo_plagas` respecto a su relación con la tabla `lote`. Si la empresa decide dar de baja o eliminar un lote de manera definitiva porque el terreno se vendió, no tiene sentido conservar el historial detallado de insectos o plagas de ese espacio. Usar `ON DELETE CASCADE` garantiza que al borrar el lote, todas sus bitácoras de monitoreo histórico se borren en automático de la base de datos, manteniendo limpio el disco duro.***

9. Categoría toxicológica equivocada aceptada por el motor

- ***¿Por qué ninguna restricción lo detectó? Porque el campo fue estructurado con un tipo de dato de texto libre común (`VARCHAR` o `TEXT`) y se omitió programar una restricción de dominio (`CHECK`). Para MySQL, la palabra "Tóxico" y la palabra "Manzana" son cadenas de texto idénticas y válidas mientras no se le ponga una regla explícita que las filtre.***
- ***Controles a agregar en la aplicación web:***
  - ***En el Frontend (Diseño de la página): Reemplazar la casilla de escritura libre por un componente de selección cerrada, como un menú desplegable (Dropdown / `<select>`), forzando al usuario a elegir únicamente entre las cuatro opciones oficiales: I, II, III o IV.***
  - ***En el Backend (Servidor): Programar una regla de validación lógica antes de enviar el comando SQL. Por ejemplo: `if (categoria_ingresada no está en ['I', 'II', 'III', 'IV']) rechazar_formulario()`.***

10. Ingreso prematuro del operario al lote (Caso RoyaStop)

- ***¿Qué dato de la base de datos debió impedirlo? El campo `reingreso_horas` (Periodo de Reingreso) del producto químico, cruzado matemáticamente con la hora exacta de la aplicación. Si el producto estipula un bloqueo de 24 horas y el sistema detecta que la consulta ocurre apenas 10 horas después, el software sabe que el lote se encuentra en un estado de alta toxicidad ambiental.***

- ***¿Cómo lo mostrarías en la página web?***
  - ***Alerta Visual de Peligro: En el mapa o panel general de lotes de la aplicación web, el terreno afectado debe pintarse en color rojo intermitente acompañado de un ícono de calavera o advertencia (⚠).***
  - ***Cronómetro de Cuenta Regresiva: Un mensaje de texto dinámico en pantalla que indique claramente: "ACCESO DENEGADO: Lote bajo efecto químico de RoyaStop. Tiempo restante para reingreso seguro: 14 horas".***
  - ***Bloqueo de Asignaciones: En el módulo de recursos humanos de la plataforma web, el software debe deshabilitar por completo el botón de "Asignar tareas" en ese lote para cualquier trabajador, impidiendo que el administrador del campo cometa un error operativo por olvido.***

Con estas reflexiones terminas el segundo punto del taller. ¿Deseas que continuemos con el siguiente punto o laboratorio de tu guía de actividades?

1. Modelo de Datos Normalizado (3FN)

Para cumplir con la 3FN, la información se divide en 4 tablas principales y 1 tabla intermedia, asegurando que cada dato esté en su lugar y se registre una sola vez:

- ***`operarios` (3FN): Almacena la información exclusiva del trabajador. (El teléfono depende únicamente del documento, no de la aplicación).***
- ***`productos` (2FN / 3FN): Almacena los datos técnicos de los insumos. (La categoría toxicológica depende únicamente del producto).***
- ***`lotes` (3FN): Registra los terrenos de la finca.***
- ***`aplicaciones` (Tabla Intermediaria M:N): Conecta las entidades anteriores registrando la bitácora de campo (fecha, dosis real y observaciones atómicas).***

3. ¿Cómo soluciona este diseño las anomalías planteadas?

- ***Solución a la Anomalía de Actualización: Si el operario cambia de teléfono, solo se modifica una sola fila en la tabla `operarios`. No importa si ha realizado 500 aplicaciones, el dato no estará duplicado de forma masiva.***
- ***Solución a la Anomalía de Inserción: Ahora puedes contratar y registrar a un operario nuevo en la tabla `operarios` sin necesidad de asignarle una aplicación de inmediato. El sistema lo permite porque las tablas están desacopladas.***
- ***Solución a la Anomalía de Borrado: Si cancelas o borras una fila de la tabla `aplicaciones` porque se digitó mal, los datos del producto no se pierden. El insumo permanece guardado de forma segura en la tabla `productos` para futuras siembras.***

6. Dependencia de `categoria_tox` y Forma Normal violada

- **De qué depende realmente:** Depende única y exclusivamente del **producto** (o insumo químico), no de la aplicación. Un fungicida no cambia su nivel de toxicidad por el hecho de aplicarse hoy o mañana, ni por aplicarse en el lote A o en el lote B.
- **Qué forma normal violaba:** Violaba la **Segunda Forma Normal (2FN)**. La 2FN exige que todos los atributos que no forman parte de la llave primaria dependan de la llave completa de la tabla. Al estar en una planilla única donde la llave implícita suele ser el identificador de la aplicación, la categoría toxicológica presentaba una *dependencia parcial*, ya que solo necesitaba saber cuál era el producto para determinarse.

7. Análisis de la columna `observaciones` y la 1FN

- **¿Cumple la 1FN? No.** La Primera Forma Normal (1FN) exige atomicidad; es decir, que cada celda contenga un único valor simple y que no existan listas ni textos compuestos que mezclen múltiples variables de negocio en una sola cadena de caracteres. El texto *"cogollero en 30% de plantas"* mezcla el nombre de la plaga con una métrica de incidencia.
- **Cómo rediseñarla para consultas analíticas:** Se debe crear una tabla independiente de **`monitoreo_plagas`** o **`evaluaciones_campo`** con campos numéricos y categóricos atómicos:

8. Cuándo es aceptable desnormalizar a propósito

Es completamente aceptable en sistemas de **Minería de Datos, Data Warehouses (OLAP) o Dashboards de alta concurrencia**.
Si una página web muestra un reporte gerencial consolidado (como el gasto total por departamento o el inventario crítico global) y es consultado miles de veces al día por múltiples usuarios, calcular miles de `JOIN`, `SUM` y `GROUP BY` en tiempo real congelaría el servidor. En ese escenario se prefiere desnormalizar creando una tabla resumen precalculada o usando vistas indexadas/materializadas. El costo es almacenar datos repetidos, pero el beneficio es una velocidad de carga instantánea en la web.

Quinto Punto: Visión Artificial con YOLO y MySQL

Para transformar las coordenadas flotantes que entrega el dron en recomendaciones agronómicas reales, lo primero que debemos hacer es diseñar una tabla optimizada en MySQL capaz de recibir ráfagas de datos masivas provenientes de los scripts de inferencia (generalmente escritos en Python).

1. Sentencia DDL: Tabla para almacenar detecciones de YOLO

Cada vez que el dron procesa un fotograma y detecta una anomalía fitosanitaria, el script insertará una fila en esta estructura:



2. Inteligencia del Negocio: Filtro por Vista (Umbral de Daño Económico)

Los drones detectan miles de cosas, pero los agrónomos solo actúan cuando la densidad de la plaga supera un peligro real. Esta vista consolida el estado actual de los lotes basándose únicamente en detecciones con alta confianza (> 85%), calculando cuántos focos de infección tiene cada terreno en su último vuelo:

Si ejecutas un `SELECT * FROM vista_alertas_fitosanitarias;` en la aplicación web, el agrónomo verá ordenadamente qué lotes específicos están invadidos por qué clases de objetos, permitiéndole prescribir el insumo químico o biológico exacto basándose en el historial unificado.

¿Deseas que programemos un **Trigger** en MySQL para que envíe de forma automática una alerta o inserte una orden de revisión si el conteo de focos de una plaga supera un límite establecido por hectárea?



```
# 0.1 Instalar e iniciar el servidor MySQL (tarda 1 a 2 minutos)
!apt-get -qq update > /dev/null
!DEBIAN_FRONTEND=noninteractive apt-get -qq install -y mysql-server > /dev/null
!service mysql start

# Crear un usuario para la clase (en producción nunca se trabaja con root)
!mysql -e "CREATE USER IF NOT EXISTS 'alumno'@'localhost' IDENTIFIED WITH mysql_native_password BY 'Colab_2026'; GRANT ALL PRIVILEGES ON *.* TO 'alumno'
!mysql --version
!pip -q install pymysql


# 0.2 Funciones de ayuda: sql() ejecuta una o varias sentencias y muestra los resultados como tabla
import pymysql, pandas as pd
from IPython.display import display

CON = pymysql.connect(host="localhost", user="alumno", password="Colab_2026",
                      autocommit=True, charset="utf8mb4")

def _dividir(script):
    # Separa sentencias por ';' respetando el texto entre comillas
    partes, actual, comilla = [], [], None
    for c in script:
        if comilla:
            actual.append(c)
            if c == comilla: comilla = None
        elif c in ("'", '"'):
            comilla = c; actual.append(c)
        elif c == ";":
            partes.append("".join(actual)); actual = []
        else:
            actual.append(c)
    partes.append("".join(actual))
    limpias = []
    for p in partes:
        lineas = [l for l in p.splitlines() if not l.strip().startswith("--")]
        p = "\n".join(lineas).strip()
        if p: limpias.append(p)
    return limpias

def sql(script, mostrar=True):
    # Ejecuta el script. Devuelve el DataFrame de la última consulta que produjo filas.
    CON.ping(reconnect=True)
    ultimo = None
    with CON.cursor() as cur:
        for s in _dividir(script):
            if s.upper().startswith("USE "):
                CON.select_db(s.split()[1].strip("`")); continue
            try:
                cur.execute(s)
            except pymysql.MySQLError as e:
                codigo, mensaje = e.args[0], e.args[1] if len(e.args) > 1 else ""
```



```
                print(f"ERROR MySQL {codigo}: {mensaje}\n   en la sentencia: {s[:120]}")
                return None
            if cur.description:
                cols = [d[0] for d in cur.description]
                ultimo = pd.DataFrame(cur.fetchall(), columns=cols)
                if mostrar: display(ultimo)
            elif mostrar and s.split()[0].upper() in ("INSERT", "UPDATE", "DELETE"):
                print(f"OK · {cur.rowcount} fila(s) afectada(s) · {s.split()[0].upper()}")
    return ultimo

def sql_bloque(sentencia):
    # Para TRIGGERS y procedimientos: se envían completos, sin dividir por ';'
    CON.ping(reconnect=True)
    with CON.cursor() as cur:
        cur.execute(sentencia)
    print("OK · bloque ejecutado")

sql("SELECT VERSION() AS version_mysql, NOW() AS hora_servidor")
```

```
W: Skipping acquire of configured file 'main/source/Sources' as repository 'https://r2u.stat.illinois.edu/ubuntu noble InRelease' does not seem to prov
[ OK ]
mysql  Ver 8.0.46-0ubuntu0.24.04.4 for Linux on x86_64 ((Ubuntu))
/tmp/ipykernel_2171/3074038526.py:42: DeprecationWarning: The 'reconnect' argument is deprecated. Create a new connection if you want to reconnect.
  CON.ping(reconnect=True)
```

	version_mysql	hora_servidor
0	8.0.46-0ubuntu0.24.04.4	2026-10-28 20:40:14

	version_mysql	hora_servidor
0	8.0.46-0ubuntu0.24.04.4	2026-10-28 20:40:14

PREGUNTAS SOBRE YOLO

1. Si suben el umbral de confianza de 0.25 a 0.80, ¿qué suele pasar?

A Aumentan los falsos positivos y sube el recall

B Disminuyen los falsos positivos, pero aumentan los falsos negativos

C No cambia nada porque NMS ya filtró las cajas

D Aumenta la cantidad de cajas guardadas

Correcto. Con un umbral alto solo quedan cajas muy seguras: sube la precisión y baja el recall. Muevan el control y miren VP, FP y FN.

---

2. ¿Qué hace el paso NMS?

A Cambia la clase de las cajas mal clasificadas

B Aumenta la resolución de la imagen

C Elimina cajas duplicadas de la misma clase que se solapan más que el umbral IoU

D Une todas las cajas de la imagen en una sola

Correcto. NMS conserva la caja de mayor confianza y suprime las que se solapan con ella por encima del umbral IoU.

3. ¿Por qué x, y, w, h se guardan normalizados entre 0 y 1?

A Para ocupar menos espacio en disco

B Para que sirvan con cualquier resolución de imagen

C Porque MySQL no admite números mayores que 1

D Para que la consulta sea más rápida

Correcto. Si la foto se guarda a otra resolución, multiplicar por el ancho y alto nuevos devuelve la caja correcta.

---

4. ¿Qué camino de llaves conecta una detección con el lote donde se tomó?

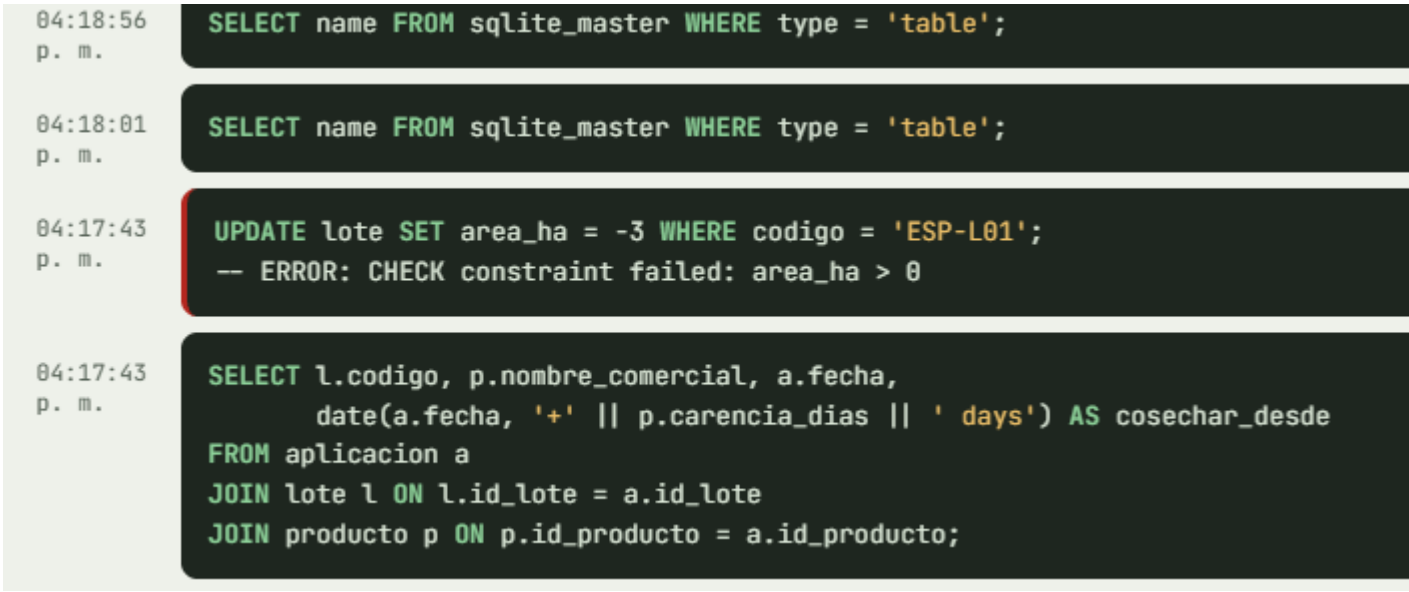
A deteccion.id_lote → lote

B deteccion.clase → clase_objetivo → lote

C deteccion.id_imagen → imagen.id_lote → lote

D No hay forma: hay que guardar el código del lote en cada detección

Correcto. La detección no repite el lote: lo obtiene por la imagen. Guardarlo dos veces crearía una dependencia transitiva.



Bloques 1 y 2: Listar las tablas de la base de datos

sql

```
SELECT name FROM sqlite_master WHERE type = 'table';
```

Usa el código con precaución.

- **Qué hace:** Esta consulta le pide al motor de la base de datos (en este caso SQLite) que muestre una lista con los **nombres de todas las tablas** que han sido creadas en el sistema.
- **Para qué sirve:** Es el comando equivalente a un "explorador de archivos". Se usa para verificar qué tablas existen antes de empezar a trabajar o insertar datos.

Bloque 3: Intento de actualización errónea (Filtro de seguridad)

sql

```
UPDATE lote SET area_ha = -3 WHERE codigo = 'ESP-L01';
-- ERROR: CHECK constraint failed: area_ha > 0
```

Usa el código con precaución.

- **Qué hace:** Intenta modificar la tabla `lote` para cambiar el tamaño del lote con código `'ESP-L01'` a un valor negativo (`-3` hectáreas).
- **Explicación del Error:** El motor de la base de datos detiene la operación y arroja un error porque la tabla tiene una regla de protección llamada **`CHECK (area_ha > 0)`**. Como físicamente no puede existir un terreno con un tamaño menor o igual a

cero, la base de datos bloquea el cambio para evitar que se guarden datos falsos o corruptos.

Bloque 4: Consulta de trazabilidad y fechas de seguridad

sql

```
SELECT l.codigo, p.nombre_comercial, a.fecha,
       date(a.fecha, '+' || p.carencia_dias || ' days') AS
cosechar_desde
FROM aplicacion a
JOIN lote l ON l.id_lote = a.id_lote
JOIN producto p ON p.id_producto = a.id_producto;
```

Usa el código con precaución.

- **Qué hace:** Junta la información de tres tablas distintas (`aplicacion`, `lote` y `producto`) para generar un reporte unificado de control de agroquímicos.
- **Explicación de sus partes:**
  - **`JOIN`**: Conecta los registros para saber exactamente en qué lote (`l.codigo`) se aplicó qué producto químico (`p.nombre_comercial`) y en qué fecha exacta ocurrió (`a.fecha`).
  - **`date(a.fecha, ...)`**: Realiza una operación matemática de tiempo. Toma la fecha en la que se fumigó el lote y le suma de forma automática los días de carencia que exige la ficha técnica de ese producto químico.
  - **`AS cosechar_desde`**: Le pone un nombre limpio a esa columna calculada para mostrarle al agrónomo en la página web la fecha exacta a partir de la cual el alimento vuelve a ser seguro para el consumo humano y puede ser recolectado.

¿Necesitas pasar estas explicaciones textuales a algún **informe de laboratorio** o requieres ayuda para escribir una nueva consulta basada en este reporte?

```sql
INSERT INTO aplicacion (id_lote, id_producto, fecha, cantidad)
VALUES (99, 1, '2026-10-14', 10);
-- ERROR: FOREIGN KEY constraint failed
```

```sql
SELECT i.id_imagen, l.codigo, d.clase, d.confianza
FROM deteccion d
JOIN imagen i ON i.id_imagen = d.id_imagen
JOIN lote l ON l.id_lote = i.id_lote
ORDER BY d.confianza DESC
LIMIT 20;
```

```sql
SELECT f.nombre AS finca, l.codigo, l.cultivo, l.area_ha
FROM lote l
JOIN finca f ON f.id_finca = l.id_finca;
```

Bloque 1: Intento de inserción erróneo (Control de integridad referencial)

sql

```
INSERT INTO aplicacion (id_lote, id_producto, fecha, cantidad)
VALUES (99, 1, '2026-10-14', 10);
-- ERROR: FOREIGN KEY constraint failed
```

Usa el código con precaución.

- **Qué hace:** Intenta registrar una bitácora de aplicación de insumos asignándola a un código de lote inexistente (`id_lote = 99`).
- **Explicación del Error:** El motor de la base de datos bloquea inmediatamente la inserción y arroja un error de **llave foránea (FOREIGN KEY)**. Esto ocurre porque el sistema protege la integridad de los datos: no permite registrar que se aplicó un químico en un terreno fantasma que no existe en la tabla principal de lotes.

Bloque 2: Consulta analítica de Visión Artificial (YOLO)

sql

```
SELECT i.id_imagen, l.codigo, d.clase, d.confianza
FROM deteccion d
JOIN imagen i ON i.id_imagen = d.id_imagen
JOIN lote l ON l.id_lote = i.id_lote
```

```
ORDER BY d.confianza DESC
LIMIT 20;
```
Usa el código con precaución.

- **Qué hace:** Cruza los datos de las detecciones de inteligencia artificial con las fotos tomadas y los lotes del terreno para mostrar un reporte de los hallazgos fitosanitarios más certeros del dron.
- **Explicación de sus partes:**
  - **`JOIN`**: Une la tabla de detecciones (`d`), con las imágenes del dron (`i`) y los lotes (`l`) para mapear con precisión qué plaga o anomalía (`d.clase`) se encontró y en qué terreno específico ocurrió (`l.codigo`).
  - **`ORDER BY d.confianza DESC`**: Ordena los resultados de mayor a menor probabilidad, dejando arriba las detecciones de las que el modelo de IA está más seguro (evitando falsos positivos).
  - **`LIMIT 20`**: Limita la salida en la pantalla de la aplicación web a únicamente los 20 registros más importantes.

Bloque 3: Consulta general de inventario de campos

sql

```
SELECT f.nombre AS finca, l.codigo, l.cultivo, l.area_ha
FROM lote l
JOIN finca f ON f.id_finca = l.id_finca;
```
Usa el código con precaución.

- **Qué hace:** Genera un listado consolidado y limpio de todos los lotes de la empresa agropecuaria, indicando a qué propiedad pertenecen.
- **Explicación de sus partes:**
  - **`JOIN finca f ON...`**: Vincula cada lote con su respectiva finca mediante el código de relación.
  - **`f.nombre AS finca`**: Renombra temporalmente la columna del nombre de la finca para que en la interfaz o reporte web se lea simplemente como un encabezado claro llamado "finca". Muestra de manera organizada el nombre de la hacienda, el código técnico del lote, qué planta hay sembrada (`l.cultivo`) y cuántas hectáreas mide (`l.area_ha`).

Con esto ya tienes explicadas las bitácoras y las consultas del dron. ¿Deseas que revisemos el código de las **vistas o triggers** del simulador para el siguiente punto?

```sql
SELECT f.nombre AS finca, l.codigo, l.cultivo, l.area_ha
FROM lote l
JOIN finca f ON f.id_finca = l.id_finca;
```

```sql
SELECT nombre_comercial, tipo, categoria_tox, carencia_dias FROM producto;
```

```sql
SELECT name FROM sqlite_master WHERE type = 'table';
```

```sql
SELECT codigo, cultivo, area_ha
FROM lote
ORDER BY area_ha DESC;
```

```sql
-- Motor SQL listo. Base de datos agro_campo cargada.
PRAGMA foreign_keys = ON;
```

Bloque 1: Consulta de inventario unificado (Fincas y Lotes)

sql

```
SELECT f.nombre AS finca, l.codigo, l.cultivo, l.area_ha
FROM lote l
JOIN finca f ON f.id_finca = l.id_finca;
```

Usa el código con precaución.

- **Qué hace:** Une los datos de la tabla `lote` con la tabla `finca`. Muestra en una sola pantalla a qué hacienda pertenece cada lote, su código de identificación, el tipo de cultivo sembrado y su tamaño exacto en hectáreas. El comando `AS finca` renombra la columna del nombre para que el reporte web se vea ordenado y legible.

Bloque 2: Consulta de inventario técnico de la bodega

sql

```
SELECT nombre_comercial, tipo, categoria_tox, carencia_dias FROM
producto;
```

Usa el código con precaución.

- **Qué hace:** Extrae la información esencial de todos los insumos agrícolas registrados en la base de datos.
- **Para qué sirve:** Es la consulta base que alimenta el catálogo de la aplicación. Le muestra al agrónomo el nombre del producto, si es un fertilizante o fungicida (`tipo`), su franja de peligro (`categoria_tox`) y cuántos días se debe esperar antes de cosechar el alimento para evitar envenenamientos (`carencia_dias`).

Bloque 3: Explorador de base de datos

sql
```
SELECT name FROM sqlite_master WHERE type = 'table';
```
Usa el código con precaución.

- **Qué hace:** Le solicita al sistema de gestión de base de datos que imprima una lista con los nombres de todas las tablas creadas por el usuario hasta el momento. Funciona como un comando de inspección rápido para confirmar que la estructura del laboratorio se cargó correctamente.

Bloque 4: Ranking de lotes por tamaño

sql
```
SELECT codigo, cultivo, area_ha
FROM lote
ORDER BY area_ha DESC;
```
Usa el código con precaución.

- **Qué hace:** Muestra el código, el cultivo y el tamaño de los lotes, pero los organiza de forma prioritaria de mayor a menor superficie gracias a la instrucción **`ORDER BY area_ha DESC`** (orden descendente).
- **Para qué sirve:** Permite identificar de un vistazo cuáles son los terrenos más grandes de la finca para priorizar la asignación de recursos o compras de semillas y fertilizantes.

Bloque 5: Inicialización y activación de llaves foráneas

sql
```
-- Motor SQL listo. Base de datos agro_campo cargada.
PRAGMA foreign_keys = ON;
```
Usa el código con precaución.

- **Qué hace:** Es un comando de configuración interno exclusivo de SQLite.

- **Para qué sirve:** Por defecto, SQLite viene con las restricciones de integridad apagadas. Esta instrucción es **la más importante del inicio del laboratorio**, ya que **activa de forma obligatoria la validación de llaves foráneas**. Gracias a este comando, el motor rechazará de inmediato cualquier intento de registrar aplicaciones en lotes fantasmas o borrar fincas que contengan cultivos asociados. Con esto ya tienes cubiertas todas las consultas base del sistema. ¿Quieres que pasemos a revisar las **últimas preguntas del cuestionario** o necesitas apoyo con las capturas de pantalla de los resultados?